

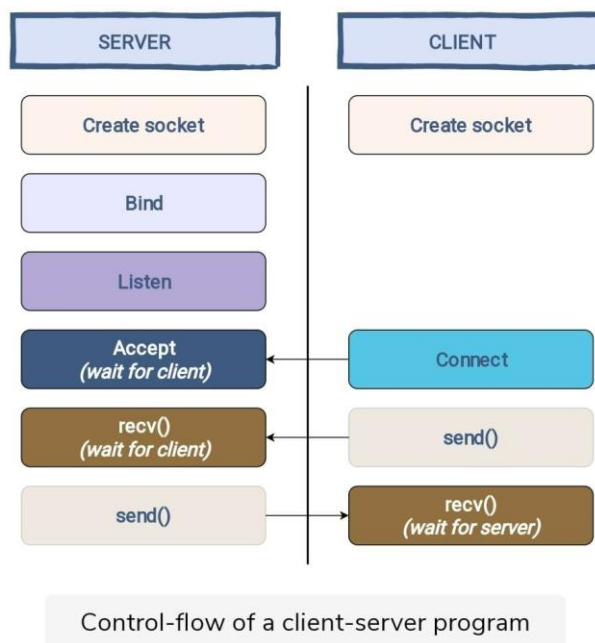
Experiment 3

Implementation of Client-Server communication using Socket Programming and TCP as transport layer protocol

Aim: Client sends a string to the server using tcp protocol. The server reverses the string and returns it to the client, which then displays the reversed string.

Description:

Steps for creating a TCP connection by a client are:



Algorithm

Steps to create a client using TCP/IP API

- Create a socket with the `socket()` system call.
- Initialize the socket address structure as per the server and connect the socket to the address of the server using the `connect()` system call.
- Receive and send the data using the `recv()` and `send()`.
- Close the connection by calling the `close()` function.

Steps to create a server using TCP/IP API

- Create a socket with the `socket()` system call.
- Initialize the socket address structure and bind the socket to an address using the `bind()` system call.
- Listen for connections with the `listen()` system call.
- Accept a connection with the `accept()` system call. This call typically blocks until a client connects to the server.
- Receive and send data by using the `recv()` and `send()`.
- Close the connection by using the `close()`.

1. Creation of client socket

```
int socket(int domain, int type, int  
          protocol);
```

This function call creates a socket and returns a socket descriptor. The domain parameter specifies a communication domain; this selects the protocol family which will be used for communication. These families are defined in <sys/socket.h>. In this program, the domain **AF_INET** is used. The socket has the indicated type, which specifies the communication semantics. **SOCK_STREAM** type provides sequenced, reliable, two-way, connection based byte streams. The **protocol** field specifies the protocol used. We always use 0. If the system call is a failure, a -1 is returned. The header files used are **sys/types.h** and **sys/socket.h**.

The header file that is to be used is

netinet/in.h

Example

```
struct sockaddr_in  
servaddr;  
servaddr.sin_family =  
AF_INET;  
servaddr.sin_port =  
htons(port_number);
```

Why htons is used ?. Numbers on different machines may be represented differently (big-endian machines and little-endian machines). In a little-endian machine the low order byte of an integer appears at the lower address; in a big-endian machine instead the low order byte appears at the higher address. Network order, the order in which numbers are sent on the internet is big-endian. It is necessary to ensure that the right representation is used on each machine. Functions are used to convert from host to network form before transmission- htons for short integers and htonl for long integers.

The value for servaddr.sin_addr is assigned using the following function

inet_pton(AF_INET, "IP_Address", &servaddr.sin_addr);

The binary value of the dotted decimal IP address is stored in the field when the function returns.

2. Binding of the client socket to a local port

This is optional in the case of client and we usually do not use the *bind* function on the client side.

3. Connection of client to the server

A server is identified by an IP address and a port number. The connection operation is used on the client side to identify and start the connection to the server.

int connect(int sd, struct sockaddr * addr, int addrlen);

sd – file descriptor of local socket

addr – pointer to protocol

address of other socket addrlen

– length in bytes of address

structure

The header files to be used are

sys/types.h and sys/socket.h It returns 0

on success and -1 in case of failure.

closing the connection

The connection can be closed using the close system call

int close(int sd);

Steps for TCP Connection for server

1. Creating a listening socket

int socket(int domain, int type, int protocol);

This system call creates a socket and returns a socket descriptor. The *domain* field used is **AF_INET**. The socket type is **SOCK_STREAM**. The **protocol** field is 0. If the system call is a failure, a -1 is returned. Header files used are **sys/types.h** and

sys/socket.h.

2. Binding to a local port

int bind(int sd, struct sockaddr * addr, int addrlen);

This call is used to specify for a socket the protocol port number where it will wait for messages. A call to *bind* is optional on the client side, but required on the server side. The first field is the *socket* descriptor of local socket. Second is a pointer to protocol address structure of this socket. The third is the length in bytes of the structure referenced by *addr*. This system call returns an integer. It is 0 for success and -1 for failure. The header files are **sys/types.h** and **sys/socket.h**.

3. Listening on the port

The listen function is used on the server in the connection oriented communication to prepare a socket to accept messages from clients.

int listen(int fd, int qlen);

fd – file descriptor of a socket that has already been bound

qlen – specifies the maximum number of messages that can wait to be processed by the server while the server is busy servicing another request. Usually it is taken as 5. The header files used are *sys/types.h* and *sys/socket.h*. This function returns 0 on success and -1 on failure.

4. Accepting a connection from the client

The accept function is used on the server in the case of connection oriented communication to accept a connection request from a client.

int accept(int fd, struct sockaddr * addressp, int * addrlen);

The first field is the descriptor of server socket that is listening. The second parameter *addressp* points to a socket address structure that will be filled by the address of calling client when the function returns. The third parameter *addrlen* is an integer that will contain the actual length of address structure of client. It returns an integer that is a descriptor of a new socket called the connection socket. Server sockets send data and read data from this socket. The header files used are *sys/types.h* and *sys/socket.h*.

Implement client server communication using TCP Socket

Tcp Server

```
# include<stdio.h>
# include<string.h>
# include<sys/socket.h>
# include<stdlib.h>
# include<netdb.h>

int main()
{
    int serversocket,clientsocket,port;
    struct sockaddr_in serveraddr,clientaddr;
    socklen_t len;
    char message[50];
    struct serveraddr;

    serversocket=socket(AF_INET,SOCK_STREAM,0);
    bzero((char*)&serveraddr,sizeof(serveraddr));
    serveraddr.sin_family=AF_INET;
    printf("Enter the port number ");
    scanf("%d",&port);
    serveraddr.sin_port=htons(port);
    serveraddr.sin_addr.s_addr=INADDR_ANY;
    bind(serversocket,(struct sockaddr*)&serveraddr,sizeof(serveraddr));
    bzero((char*)&clientaddr,sizeof(clientaddr));
    len=sizeof(clientaddr);
    listen(serversocket,5);
    printf("\nWaiting for client connection\n");printf("\nhai:");
    clientsocket=accept(serversocket,(struct sockaddr*)&clientaddr,&len);
    printf("\nClient connectivity received.\n");
    printf("\nReading message from the client.\n");
```

```

read(clientsocket,message,sizeof(message));
printf("\nThe client has sent.%s",message);
printf("\nSending message to the client.\n");
write(clientsocket,"YOUR MESSAGE RECEIVED.",sizeof("YOUR MESSAGE
RECEIVED."));
close(clientsocket);
close(serversocket);
}

```

Tcp Client

```

#include<stdio.h>
#include<sys/types.h>
#include<netinet/in.h>
#include<netdb.h>
#include<strings.h>
int main()
{
    int clientsocket,port;
    struct sockaddr_in serveraddr;
    socklen_t len;
    char message[50];
    clientsocket=socket(AF_INET,SOCK_STREAM,0);
    bzero((char*)&serveraddr,sizeof(serveraddr));
    len=sizeof(serveraddr);
    serveraddr.sin_family=AF_INET;
    printf("Enter the port number ");
    scanf("%d",&port);
    serveraddr.sin_port=htons(port);
    printf("\nTrying to connect to the server.\n");
    connect(clientsocket,(struct sockaddr*)&serveraddr,sizeof(serveraddr));

```

```
printf("\nConnected to the server.\n");
printf("\nSending message for server connection");
send(clientsocket,"HI,IAM CLIENT...",sizeof("HI,IAM CLIENT..."),0);
printf("\nReceiving message from server.\n");
recv(clientsocket,message,sizeof(message),0);
printf("\nMessage received.\t%s\n",message);
close(clientsocket);
}
```

OUTPUT

SERVER

```
gcc tcpnews.c -o server
```

```
./server
```

Enter the port number 5016

Waiting for client connection

hai:

Client connectivity received.

Reading message from the client.

The client has sent. HI,I AM CLIENT...

Sending message to the client.

CLIENT

```
gcc tcpnews.c -o client
```

```
./client
```

Enter the port number 5016

Trying to connect to the server.

Connected to the server.

Sending message for server connection

Receiving message from server.